

Web Applications

JavaScript Promises and async/await

Suhel Hammoud

■ Promises and async/await in JavaScript (ES2015+)

In this lecture, you'll learn how JavaScript handles asynchronous operations using Promises and the async/await syntax introduced in ES2017.

Why Asynchronous Programming?

- JavaScript is **single-threaded**.
- Blocking operations (e.g., network requests, file access) freeze the UI or delay execution.
- We use async constructs to handle long-running operations **non-blockingly**.

Real-world example:

```
const data = fetch('https://api.example.com');  
console.log('This runs immediately!');
```

```
const data = fetch('https://jsonplaceholder.typicode.com/posts');  
console.log('This runs immediately!');
```

`fetch()` returns a Promise immediately.

The browser continues execution without waiting.

Only when you await or `.then()` the result do you get the response later.

What is a Promise?

A **Promise** represents the eventual result (or error) of an asynchronous operation.

It can be in one of three states:

-  Pending
-  Fulfilled
-  Rejected

❌ Creating and Using Promises

```
const fetchData = new Promise((resolve, reject) => {
  setTimeout(() => {
    resolve("Data received!");
  }, 1000);
});

fetchData.then((result) => {
  console.log(result);
}).catch((err) => {
  console.error(err);
});
```

✅ Use `.then()` for success

❌ Use `.catch()` for failure

■ Chaining Promises

```
fetch('https://api.example.com')  
  .then(res => res.json())  
  .then(data => console.log(data))  
  .catch(err => console.error(err));
```

- Each `.then()` returns a new Promise
- Enables powerful sequential logic
- Errors propagate through `.catch()`

Common Mistake: Callback Hell

```
setTimeout(() => {  
  console.log('Step 1');  
  setTimeout(() => {  
    console.log('Step 2');  
    setTimeout(() => {  
      console.log('Step 3');  
    }, 1000);  
  }, 1000);  
}, 1000);
```

😞 This is hard to read and debug.

Common Mistake: Callback Hell

✓ Promises to solve this.

```
function delay(ms) {  
  return new Promise(resolve => setTimeout(resolve, ms));  
}  
  
delay(1000)  
  .then(() => {  
    console.log('Step 1');  
    return delay(1000);  
  })  
  .then(() => {  
    console.log('Step 2');  
    return delay(1000);  
  })  
  .then(() => {  
    console.log('Step 3');  
  });
```

Common Mistake: Callback Hell

✓ Promises and async/await solve this.

```
// Helper function to delay execution
const delay = (ms) => new Promise(resolve => setTimeout(resolve, ms));

async function runSteps() {
  await delay(1000);
  console.log('Step 1');

  await delay(1000);
  console.log('Step 2');

  await delay(1000);
  console.log('Step 3');
}

runSteps();
```

What is async/await?

- Introduced in ES2017 (ES8)
- Syntactic sugar over Promises
- Makes async code look like sync code

```
async function myFunc() {  
  const res = await fetch('https://api.example.com');  
  const data = await res.json();  
  console.log(data);  
}
```

 Clean, readable, maintainable

async Keyword

- Declares a function that always returns a Promise
- Can use `await` inside it

```
async function getUser() {  
  return 'John Doe';  
}  
  
getUser().then(name => console.log(name)); // 'John Doe'
```

await Keyword

- Pauses execution **until a Promise settles**
- Only valid inside `async` functions

```
async function fetchData() {  
  const res = await fetch('https://api.example.com');  
  const data = await res.json();  
  console.log(data);  
}
```

 `await` does **not block the main thread**, only the function scope

Error Handling with async/await

```
async function loadData() {  
  try {  
    const res = await fetch('https://api.example.com/404');  
    const data = await res.json();  
    console.log(data);  
  } catch (error) {  
    console.error('Error:', error);  
  }  
}
```

 Always wrap `await` in `try/catch` to catch rejections

Parallel Execution with Promise.all()

```
async function loadParallel() {  
  const [a, b] = await Promise.all([  
    fetch('/api/a').then(r => r.json()),  
    fetch('/api/b').then(r => r.json())  
  ]);  
  console.log(a, b);  
}
```

- Runs both fetches **concurrently**
- More efficient than waiting sequentially

When to Use Promises vs async/await

Situation	Use
Simple chaining	<code>.then().catch()</code>
Sequential logic	<code>async/await</code>
Multiple tasks at once	<code>Promise.all()</code>
Clean error handling	<code>try/catch</code>
Complex conditional flow	<code>async/await</code>

Use **async/await** for clarity, especially in complex flows.

Summary

- Promises are objects representing future values.
- `async` declares a function that returns a Promise.
- `await` pauses execution until a Promise settles.
- Use `try/catch` for error handling.
- Use `Promise.all()` to run tasks in parallel.